CSC 4553

Fall 2026 - Day 15

Admin

- Lab due Friday, 18:00
- Quiz due Monday, 18:00
- Sign up for code review slots
  https://calendar.app.google/XGVki6gTCicVj8BU9
- Mid-semester survey on Canvas

Handling deadlocks

Questions to consider

- What are the approaches to handling deadlock? How practical is each? How do they differ in approach?
- How does deadlock avoidance differ from deadlock prevention? What techniques do we implement for deadlock avoidance?
- What are the trade-offs between detecting/recovering from deadlock versus preventing or avoiding it, and when is detection practical?

How should the OS handle deadlocks?

- OS can **prevent** deadlock: ensure deadlocks can never occur (by preventing one of the deadlock requirements from occurring)
- OS can **avoid** deadlock: like prevention, but given more information about a process's resource needs
- OS can **detect** deadlock: allow system to deadlock, detect and recover
- OS can **do nothing**, and let applications handle it
  - → **Ostrich Algorithm**: head in sand; wait

Deadlock prevention

- Remember, we just have to break one of these to prevent deadlock
  - → Mutual exclusion
  - → Hold and wait
  - → No preemption
  - → Circular wait
- Any ideas?

Attacking mutual exclusion

- Make resources sharable so they aren't held by only one process at a time; some resources are naturally sharable (e.g., read-only files), but preventing mutual exclusion is fundamentally difficult as so many resources are non-sharable: printers, writable files, mutex locks, etc.

  → Spoolers can sidestep this for some resources (e.g., a print spooler accepts jobs so no one "owns" the printer directly)

Attacking hold and wait

- When a process requests a resource, it cannot be holding any others
- A process must request **all** of its resources at one time
- See any downsides?
  - → Must know all needed locks a priori
  - → Will likely decrease concurrency
  - → Poor utilization: resources held long before they're used
  - → Starvation: a process needing many popular resources may never get them all at once

Attacking no preemption

- If a process holding resources requests one that can't be granted, it releases everything it holds
- Restarted only when it can regain its old resources plus the new
- Downsides?
  - → **Livelock**: processes repeatedly release and retry, but none make progress
  - → Only works for resources whose state can be saved/restored
  - → Must roll back any partial work before releasing
  - → Starvation is possible

Attacking circular wait

- How could you prevent by focusing on the circular wait requirement?
- Impose a total ordering of all resource types
  - → For example, resource types R1, R2, …, Rn have ordering R1 < R2 < … < Rn
  - → A process that needs multiple resources must request them in increasing order of enumeration
- Downsides?
  - → May be difficult to impose a meaningful ordering
  - → May lead to reduced concurrency

Fix the transfer

```
1 void transfer(account *a, account *b, int amt) {
2   pthread_mutex_lock(&a->lock);
3   pthread_mutex_lock(&b->lock);
4   a->balance -= amt;
5   b->balance += amt;
6   pthread_mutex_unlock(&b->lock);
7   pthread_mutex_unlock(&a->lock);
8 }
```

- **Think**: Rewrite `transfer()` so it can never deadlock. Think of different answers for each of: breaking no preemption, hold and wait, and circular wait.
- **Discuss**: Compare with a neighbor. Then extend it: `transfer_many(account *accts[], int n)` must lock all `n` accounts before moving money.

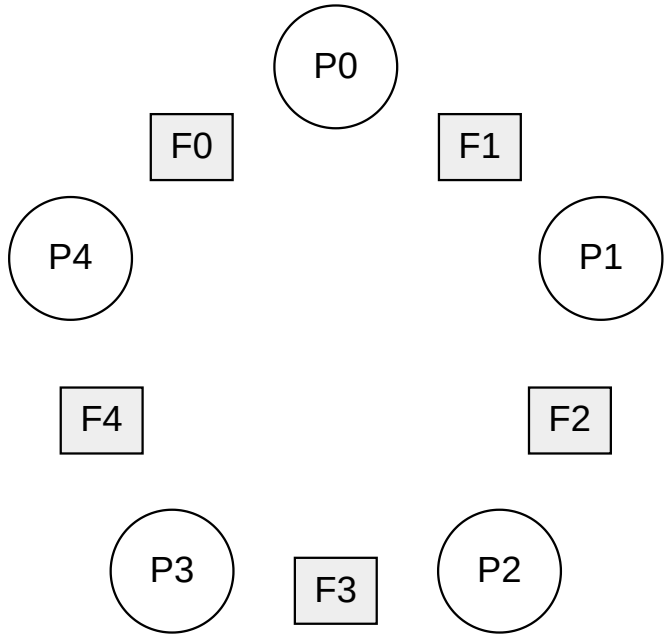
Deadlock prevention summary

- Which approach is best?
  - → Attacking mutual exclusion is generally not feasible
  - → Attacking hold and wait is wasteful and requires prescience
  - → Attacking no preemption is difficult due to non-preemptable resources, and risks livelock
  - → Attacking circular wait is practical but may reduce concurrency - often the best choice and most used

Dining philosophers

- Five philosophers around a table, one fork between each pair
- To eat, a philosopher needs **both** adjacent forks

```
1  void philosopher(int i) {
2    while (1) {
3      think();
4      lock(fork[i]);             // left
5      lock(fork[(i + 1) % 5]);   // right
6      eat();
7      unlock(fork[(i + 1) % 5]);
8      unlock(fork[i]);
9    }
10 }
```



Break one condition

- **Think**: How can the philosophers deadlock? Point to each of the four conditions in the code.
- **Discuss**: For each condition, explain how it can be broken. Change the code (or the rules) so deadlock is impossible.
  - → Mutual exclusion
  - → Hold and wait
  - → No preemption
  - → Circular wait
- Also consider the **cost** of your fixes

```
1  void philosopher(int i) {
2    while (1) {
3      think();
4      lock(fork[i]);          // left
5      lock(fork[(i+1) % 5]); // right
6      eat();
7      unlock(fork[(i+1) % 5]);
8      unlock(fork[i]);
9    }
10 }
```

What isn’t clear?

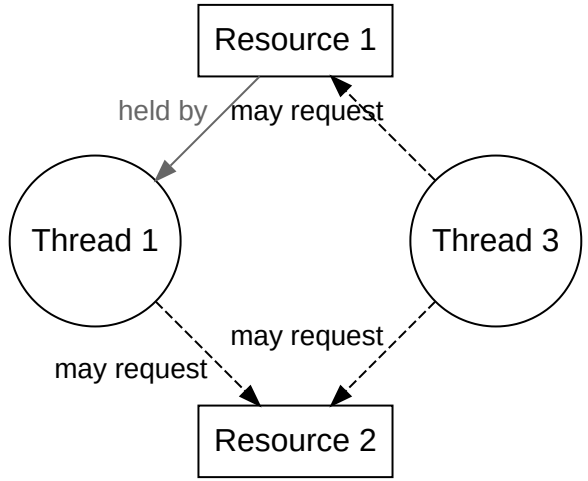
Comments? Thoughts?

Deadlock avoidance

- All of the prevention techniques can be heavy-handed, reducing concurrency and overall system throughput
- What if we had more information about process resource needs before allocation?
  - → Could we make smarter decisions about resource allocation?
- This is the idea behind deadlock avoidance

Avoidance via resource-allocation graph

- Introduce the idea of a **claim edge** (dashed line) from a process to a resource
  - → Indicates that the process may request that resource in the future
- The system can maintain this graph, and only grant resources that will not create a cycle
  - → e.g., if T3 requests R2, granting it would create the cycle T1 ⇢ R2 → T3 ⇢ R1 → T1, so it must wait
- Any downsides / limitations?
  - → Overhead of maintaining the graph and checking for cycles
  - → Only applicable to single instance resources



Deadlock avoidance in practice

- Dijkstra's Banker's algorithm is the textbook example — each process declares its max resource needs, and requests are only granted if the resulting state is "safe." Not used in practice: overhead plus the prescience requirement
- Resource-allocation graph with claim edges sees occasional use in constrained settings (e.g., embedded systems where needs are predictable)

Detecting and recovering from deadlock

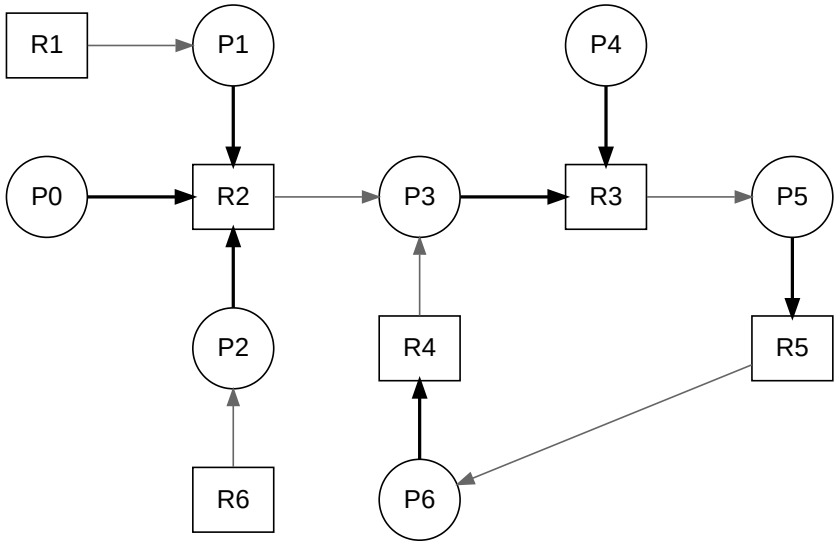
- Allow deadlocks to occur, then detect and recover
- In some cases, recovery is pretty simple. If your machine froze once per year (maybe it does?) what would you do?
  - → Reboot
- What scenarios do you think complex detection and recovery would make sense?
  - → Systems with high availability requirements where manual intervention is costly or impractical
    - telecommunications systems, financial transaction systems, healthcare systems
    - critical infrastructure systems

Detection

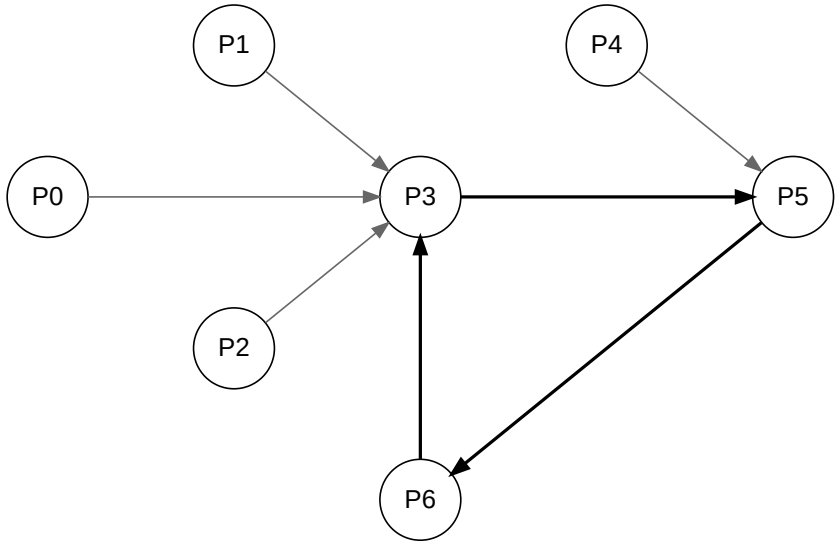
- Use a wait-for graph: a simplified resource-allocation graph that removes resource nodes
  - → An edge from process P_i to P_j indicates that P_i is waiting for a resource held by P_j
  - → A cycle in this graph indicates a deadlock
  - → $O(n^2)$ algorithm to search for cycles (beyond the cost of maintaining the graph itself)
  - → Only applies to single instance resources

Detection (cont'd)

Resource-allocation graph



Wait-for graph



Detection (cont'd)

- For multiple instance resources, we can use an algorithm similar to the Banker's algorithm to detect deadlock
- Question: How often should we run the detection algorithm?
  - → Too often: wasteful overhead
  - → Not often enough: long delays before recovery
  - → What if we run it when a process requests a resource?
    - Could allow us to assign blame for deadlock to the requesting process
  - → Heuristically (CPU utilization, etc.)
    - Less overhead, but harder to assign blame

Recovery options

- Process termination
  - → Abort all deadlocked processes (brute force)
  - → Abort one process at a time until deadlock is resolved
    - How to choose which process to abort?
      - Priority, time running, resources used, etc.
- Resource preemption
  - → Temporarily take resources away from some processes and give to others until deadlock is resolved
  - → Highly dependent on the resource type
  - → Rollback problem - processes may need to be rolled back to a safe state before resources can be reallocated
  - → Starvation problem - same victims may be chosen repeatedly

Pick the policy

- For each system below, which deadlock strategy fits best — and **why**?
  1. Flight control software on a commercial aircraft
  2. A university course-registration system during peak enrollment
  3. Your personal laptop running a browser, editor, and Slack
- For your pick, name the cost you're accepting (lost concurrency? overhead? user-visible failures?)

What isn’t clear?

Comments? Thoughts?